학습 목표

1. **하네스, 하네스 엔지니어링, 평가 하네스의 차이를 예시로 설명한다**
   정의 범위와 측정 관계까지 구분한다

2. **AI 프로그램 구성도에서 모델과 하네스 구성요소를 구분해 표시한다**
   루프, 도구, 컨텍스트, 상태, 가드레일, 관측, 비용, 평가

3. **가이드/센서, 계산적/추론적 통제의 틀로 하네스 장치를 분류한다**

4. **워크플로와 에이전트 중 과제에 맞는 방식을 근거를 들어 고른다**

5. **SDK 버전을 고정한 실습 환경에서 첫 API 호출의 request_id와 usage를 확인한다**
   uv, anthropic, claude-agent-sdk, pydantic, pydantic-ai, pytest

무단전재 및 배포 금지

데모는 되는데 운영에서는 안 된다

열 줄짜리 사내 문서 Q&A 봇이 실제 사용에서 드러낸 실패들

환각
- 연차 규정을 묻자 없는 규정을 답함
- 그럴듯해서 사용자가 믿기 쉬움
- 막을 장치: 검색 주입 + 인용 검사(3, 5장)

파싱 실패
- JSON 앞에 설명 문장이 붙음
- 화면 연동 코드의 파싱이 깨짐
- 막을 장치: pydantic 형식 검증(5장)

답변 잘림
- 긴 절차 답이 중간에 끊김
- 잘린 사실을 아무도 모름
- 막을 장치: stop_reason 검사(2장)

가짜 행동
- '생성했습니다'라고 답함
- 실제로 만든 티켓은 없음
- 막을 장치: 실제 도구 실행기(3장)

추적 불가
- 요금 청구서만 도착함
- 질문별 비용 기록이 없음
- 막을 장치: JSONL 로그, 비용 기록(9장)

해법의 위치
- 검색 주입, 형식 검사, 잘림 확인
- 도구 실행, 호출 기록
- 모두 모델 바깥의 코드
- 2~9장에서 하나씩 구현

> ***하네스(harness): 말의 힘을 원하는 방향으로 쓰게 하는 마구***

무단전재 및 배포 금지

같은 모델, 다른 하네스, 다른 결과

하네스는 성능과 비용을 함께 끌어올린다

하네스가 결과를 좌우한 근거 사례

사례	하네스 변화	결과
Anthropic 멀티에이전트 리서치	단일 → 오케스트레이터-워커	내부 eval에서 +90.2% 성능
Meta-Harness 논문	하네스 코드만 자동 탐색	텍스트 분류 +7.7점, 토큰 4배 절감
Anthropic 레트로 게임 메이커	단독 → 계획, 생성, 평가 분리	20분/$9 → 6시간/$200
Anthropic 토큰 사용량 측정	채팅 → 에이전트 → 멀티	에이전트 약 4배, 멀티에이전트 약 15배

1. Meta-Harness: 수학 RAG에서 보지 않은 모델 5종 평균 +4.7점
2. 게임 메이커: $9 결과는 핵심 기능 고장, $200 결과는 완성도 높음
3. 하네스는 잘 되게 하는 장치이자 쓸 금액을 통제하는 장치

> ***성능이 오르면 비용도 오른다: 예산 통제도 하네스의 기본 기능***

출처: Anthropic 엔지니어링 블로그(멀티에이전트 리서치 시스템, 장시간 앱 하네스 설계), Meta-Harness 논문(arXiv 2603.28052)

무단전재 및 배포 금지

에이전트 = 모델 + 하네스

LangChain Trivedy: 모델이 아니면 전부 하네스다

모델

- 다음 토큰을 예측함
- 텍스트 또는 tool_use 요청을 냄
- model= 설정과 호출 코드는 하네스 쪽

하네스

- 프롬프트와 도구 정의를 모델에 전달
- 상태, 도구 실행, 피드백 루프, 제약 제공
- 구성: 루프, 도구, 컨텍스트, 검증
- 권한, 재시도, 상태, 관측, 비용

하네스 엔지니어링

- 바깥 구조를 의도적으로 설계
- 테스트하고 계속 고쳐 나감
- 모델은 그대로 두고 신뢰성, 안전, 비용 개선

교재 기본 정의

- 에이전트 하네스 = 모델 바깥 런타임 구조
- 루프, 도구 실행기, 검증기, 권한 검사
- 재시도, 체크포인트, 로그, 예산 카운터
- 모두 파이썬으로 직접 만드는 부분

> ***사용자 요청 → 하네스 ⇄ 모델 → 결과, 부작용, 로그***

출처: Vivek Trivedy, The Anatomy of an Agent Harness (LangChain 블로그)

무단전재 및 배포 금지

출처마다 다른 하네스의 정의

2026년 초 빠르게 퍼져 범위와 기원 설명이 엇갈린다

출처별 정의 비교

출처(시기)	정의의 초점	범위
Hashimoto (2026-02)	같은 실수를 막는 엔지니어링 습관	실천 중심, 코딩 에이전트 사용자
LangChain (2026-03)	모델을 뺀 모든 것	가장 넓음
Böckeler (2026-04)	사용자가 프로젝트에 씌우는 바깥 층	사용자 측 바깥 하네스
OpenAI Codex 팀 (2026-02)	환경 설계, 의도 명세, 피드백 루프	대규모 저장소 운영(2차 보도)
Anthropic evals (2026-01)	에이전트 하네스와 평가 하네스 구분	한 단어가 두 뜻으로 쓰임
Meta-Harness (2026-03)	모델이 볼 것을 통제하는 코드	컨텍스트 엔지니어링에 가까움

1. InfoQ 의역: 컨텍스트 엔지니어링은 무엇을 볼지, 하네스 엔지니어링은 무엇을 막고 측정하고 교정할지
2. OpenAI 원문은 미확인이라 직접 표현인지 단정하지 않음
3. Fowler 사이트와 LangChain은 컨텍스트 엔지니어링을 하위 영역으로 봄: 경계는 미합의

> ***이 교재의 하네스 = 모델 바깥의 런타임 구조***

출처: Hashimoto 블로그, LangChain 블로그, martinfowler.com(Böckeler), InfoQ(2026-02), Anthropic evals 글, arXiv 2603.28052

무단전재 및 배포 금지

에이전트 하네스와 평가 하네스

한 단어가 두 뜻: 동작시키는 하네스와 측정하는 하네스

두 하네스의 구분(Anthropic evals 글 기준)

구분	에이전트 하네스(scaffold)	평가 하네스
하는 일	모델을 에이전트로 동작시킴	eval 실행, 기록, 채점, 집계
도는 시점	운영 중 매 요청마다	개발 중, CI, 모델 교체 시
파이썬 예	run_agent 루프, pydantic 검증	pytest, grader, pass^k 집계
다루는 장	2~9장	10~11장

1. pass^k: k번 시도해 k번 모두 성공할 확률(일관성)
2. pass@k: k번 중 한 번 이상 성공할 확률(10장에서 자세히)
3. eval 점수는 언제나 모델 + 에이전트 하네스 조합의 점수

> ***평가 하네스로 측정하고, 그 결과로 에이전트 하네스를 고친다***

출처: Anthropic, Demystifying evals for AI agents

무단전재 및 배포 금지

하네스 엔지니어링의 형성 과정

이름은 최근이지만 기법은 몇 년에 걸쳐 쌓였다

1. **2024-12 에이전트 패턴 정착**
   Building effective agents: 체이닝, 라우팅, 병렬화 등 패턴 정리. 가장 단순한 해법, API 직접 호출부터

2. **2025-06 멀티에이전트 리서치**
   오케스트레이터-워커 구조로 +90.2% 성능, 토큰은 약 15배

3. **2025-09 컨텍스트 엔지니어링**
   컨텍스트 부패 대응: 컴팩션, 구조화 노트, 서브에이전트, JIT 검색

4. **2025-11 장시간 에이전트**
   feature list JSON, progress 파일, git 커밋으로 세션 간 기억을 이음

5. **2026-02~04 용어 확산**
   Hashimoto, OpenAI 사례 보도, LangChain, Böckeler 글로 분야 이름이 됨

6. **2026-03 자동 최적화**
   Meta-Harness: 코딩 에이전트가 이전 후보의 코드, 점수, 트레이스를 읽고 하네스를 고침(11장)

출처: Anthropic, Building effective agents; Effective context engineering for AI agents; Effective harnesses for long-running agents

무단전재 및 배포 금지

워크플로와 에이전트 고르기

가장 단순한 해법에서 시작해 위에서부터 묻고, 처음 예가 나오는 곳에서 멈춘다

워크플로와 에이전트 비교

기준	워크플로	에이전트
제어 흐름	개발자의 파이썬 코드	모델의 tool_use 결정
예측 가능성	높음, 단계 수 고정	낮음, 턴 수 가변
비용과 지연	상한 계산이 쉬움	정지 조건이 꼭 필요
잘 맞는 과제	분류, 추출, 요약	탐색, 조사, 다단계 작업
하네스 부담	검증, 재시도 중심	루프, 정지 조건, 권한, 상태

1 단발 호출 + 프롬프트 + 검색 주입으로 충분하면 거기서 끝

2 단계를 미리 나열할 수 있으면 워크플로

3 실행 중에만 알 수 있으면 에이전트 + 정지 조건(2장)

> ***예제는 워크플로(분류)와 에이전트(검색 후 티켓)를 섞는다***

출처: Anthropic, Building effective agents

무단전재 및 배포 금지

하네스를 분류하는 두 축

모델이 결정하기 전이면 가이드, 결정을 보고 판정하면 센서

가이드/센서와 계산적/추론적 통제(2×2)

통제	가이드(결정 전)	센서(결정 후)
계산적	strict 도구 스키마, 출력 스키마, 고정된 워크플로 라우팅, 도구 허용 목록(delete_* 미노출)	pydantic 검증기, 인용 ID 검사, max_tokens 검사, 실행기 허용 목록, PreToolUse 훅, 예산 카운터, pytest 회귀 eval
추론적	시스템 프롬프트, 도구 docstring, few-shot 예시, AGENTS.md	LLM-as-judge, AI 코드 리뷰, 회의적으로 튜닝한 평가자 에이전트

1. 계산적: 결정적, 빠름, 저렴 → 먼저 쓴다 / 추론적: 의미 판단
2. 가이드만 있으면 효과를 모르고, 센서만 있으면 실수가 반복된다
3. AGENTS.md 한 줄 = 추론적 가이드, 린터와 구조 테스트 = 계산적 센서(코딩 에이전트 사용자 하네스)

> ***'삭제 금지'는 목록에서 빼고(가이드) 실행 전에 막는다(센서)***

출처: Birgitta Böckeler, Harness engineering for coding agent users (martinfowler.com)

무단전재 및 배포 금지

하네스 구성요소 지도와 로드맵

구성요소마다 막는 실패가 있고, 그것이 교재의 목차다

에이전트 하네스 구성요소(2~9장)

구성요소(장)	막아 주는 대표 실패	코드에서 알아보는 단서
루프와 정지 조건(2장)	무한 루프, 비용 폭주	max_turns, stop_reason 분기
도구 실행기(3장)	잘못된 인자, 예외로 루프 붕괴	execute_tool, tool_result
컨텍스트, 메모리(4장)	컨텍스트 부패, 기억 단절	검색 결과와 요약을 조립
구조화 출력, 검증(5장)	파싱 실패, 가짜 인용	pydantic 모델 검증
가드레일, 권한(6장)	위험한 동작, 프롬프트 인젝션	승인 프롬프트, 허용 목록, 훅
재시도(7장), 상태(8장)	일시적 API 장애, 중복 실행, 작업 손실	timeout=, 체크포인트
관측, 비용(9장)	원인 추적 불가, 예산 초과	JSONL 로그, request_id

1. 평가, 회귀(10장): eval 과제 + grader로 측정
2. 자기 개선(11장): 실패 트레이스로 하네스 수정
3. 구성요소는 모델 한계에 대한 가정(예: Opus 4.6)

> ***모델이 바뀌면 eval로 재검증하고 쓸모없는 부분은 걷어낸다***

출처: Anthropic, Harness design for long-running application development

무단전재 및 배포 금지

확인 문제: 코드에서 하네스 찾기

handle() 발췌의 (가)~(카)를 구성요소 지도에 대응시킨다

```python
def handle(q: str) -> Reply:
    msgs = build_context(q, search_docs(q))  # (가)
    for turn in range(MAX_TURNS):  # (나)
        msg = with_retry(client.messages.create,  # (다)
            model=MODEL, timeout=30, ...)  # (라)
        spent_usd += cost_of(msg.usage)  # (마)
        if spent_usd > BUDGET_USD: raise BudgetExceeded
        log_jsonl(msg._request_id, msg.usage)  # (바)
        if msg.stop_reason != "tool_use":
            return Reply.model_validate_json(...)  # (사)
        for b in tool_uses(msg):
            if b.name == "create_ticket": approve(b)  # (아)
            msgs += execute_tool(b)  # (자)
        save_checkpoint(msgs)  # (차)
```

1. 모델은 (라)의 MODEL이 가리키는 LLM뿐: 설정값과 호출 코드는 하네스
2. (다)는 with_retry와 timeout으로 API 장애에 대비 (7장)
3. (카) 회귀 테스트 3회 실행: 평가 하네스(10장)
4. (카)는 tests/test_regression.py, 나머지는 지도에 대응

> ***모델이 아닌 줄은 모두 하네스다***

무단전재 및 배포 금지

예제 프로젝트와 SDK 버전 고정

사내 문서 Q&A와 티켓 처리 에이전트를 장마다 키운다

예제와 도구

- 예제: 규정을 검색해 근거와 함께 답하고, 미해결 문제는 IT 티켓으로
- 코딩 에이전트 사용자는 안쪽 하네스(루프, 권한, 컴팩션)를 못 바꾸지만 우리는 직접 만든다 → claude-agent-sdk는 비교 기준점(2, 6, 8장)
- pydantic은 계산적 센서의 핵심, pydantic-ai는 5장 비교 대안
- 변경 리스크: SDK 메이저 버전, Agent SDK 옵션, OTel GenAI 규약(9장)

고정 전략

1. 핵심 패키지를 pyproject.toml에 ==로 정확히 고정한다
2. uv.lock을 커밋하고 CI에서는 uv sync --locked로 설치한다
3. 업그레이드는 별도 PR로, 10장 회귀 eval 통과 후 병합한다
4. 실행 로그마다 SDK 버전을 남긴다(9장)

> ***업그레이드 전 changelog 재확인: 예) output_format → output_config.format***

출처: Hashimoto 블로그, InfoQ(2026-02), Anthropic Python SDK 문서, Structured outputs 문서, Claude Agent SDK reference(2026-10 기준)

무단전재 및 배포 금지

기준선 코드: 무엇을 기록하나

단발 호출 응답에서 기록할 값을 꺼내 JSONL로 남긴다

python

```
msg = client.messages.create(
    model=MODEL, max_tokens=MAX_TOKENS,
    messages=[{"role": "user", "content": q}],
)
text = ...  # 응답의 텍스트 블록
record = {
    "request_id": msg._request_id,
    "stop_reason": msg.stop_reason,
    "usage": {"input_tokens": msg.usage.input_tokens,
              "output_tokens": msg.usage.output_tokens},
    "sdk_version": version("anthropic"),
    "auto_observation": observe(q, msg, text),
}
f.write(json.dumps(record, ensure_ascii=False) + "\n")
```

1. request_id는 보통 req_로 시작
2. Q2: stop_reason=max_tokens 예상
3. 모델 ID는 models.list()로 확인
4. version()은 importlib.metadata에서 가져옴

> ***JSONL 한 줄 = 질문 하나의 기준선 기록***

무단전재 및 배포 금지

실습 안내

하네스 없는 단발 호출로 기준선을 만들고 요구사항을 뽑는다

1 **프로젝트 생성과 버전 고정**
uv init --package로 만들고 핵심 패키지를 ==로 고정. tests/test_env.py로 고정 상태 확인

2 **하네스 없는 단발 호출**
질문 5개(Q1~Q5)를 던지고 stop_reason, usage, request_id를 runs/ch01_baseline.jsonl에 기록

3 **관찰한 실패 정리**
환각, 잘림, JSON 파싱 실패, 가짜 행동, 인젝션을 직접 읽고 관찰됨 또는 미재현으로 기입

4 **하네스 요구사항 시트 작성**
실패마다 막을 구성요소, 장, 측정 가능한 수용 기준을 적는다(미재현 행도 유지)

5 **결과 확인**
JSONL 5줄의 기준선, pytest 2개 통과, 고정을 풀면 테스트가 실패하는지 확인

> ***하네스는 운 좋은 결과가 아니라 최악의 입력을 기준으로 설계한다***

무단전재 및 배포 금지

핵심 정리

1. **Agent = Model + Harness: 모델 바깥 전체가 하네스다**
   이를 설계, 측정, 개선하는 일이 하네스 엔지니어링

2. **정의 범위는 출처마다 다르며, 교재는 모델 바깥 런타임 구조로 정의한다**

3. **에이전트 하네스는 동작시키고, 평가 하네스는 그 조합을 측정한다**
   eval 점수는 언제나 모델 + 하네스의 점수

4. **하네스는 성능과 비용을 함께 좌우한다**
   +90.2%, +7.7점, 토큰 4~15배, $9 → $200

5. **가장 단순한 해법에서 시작한다**
   단계를 나열할 수 있으면 워크플로, 아니면 정지 조건을 갖춘 에이전트

6. **가이드/센서, 계산적/추론적 두 축으로 빈 곳을 찾는다**
   가능하면 계산적 통제를 먼저 쓴다

7. **구성요소는 모델 한계에 대한 가정이다**
   모델이 바뀌면 eval로 재검증하고 필요 없는 부분은 걷어낸다

8. **SDK는 ==로 고정하고 lock 파일을 커밋한다**
   업그레이드는 회귀 eval 통과 후 반영

무단전재 및 배포 금지